

MEMORIA TÉCNICA

MazeBank API

API RESTful Bancaria con Node.js, Express y PostgreSQL

Autor:

Jon Aldekoa

Fecha: Mayo 2026

Repositorio: github.com/Jaldekoa/mazebank

Índice

Índice	2
1 Introducción y Contexto	3
1.1 Antecedentes y Definición del Problema.....	3
1.2 Estado del Arte y Justificación	3
2 Análisis de Requisitos	4
2.1 Requisitos Funcionales (RF).....	4
2.2 Requisitos No Funcionales (RNF).....	4
3 Planificación y Gestión del Tiempo	6
3.1 Desglose de Tareas (Backlog).....	6
3.2 Cronograma de Trabajo (Gantt).....	6
4 Análisis Funcional y Flujos	8
4.1 Flujo de Transferencia (Diagrama de Secuencia).....	8
4.2 Modelo de Datos (Entidad-Relación)	8
5 Arquitectura y Modelo de Datos	9
5.1 Arquitectura MVC	9
5.2 Diseño de la API (Endpoints Principales).....	9
5.3 Stack Tecnológico y Justificación	9
6 Retos Técnicos y Soluciones	11
6.1 Reto: Transacciones Financieras Atómicas	11
6.2 Reto: Protección del Balance con Hooks de Sequelize.....	11
7 Pruebas de Calidad (QA)	13
8 Conclusiones y Futuros Pasos	14
Mejoras para la v2.0	14

1 Introducción y Contexto

1.1 Antecedentes y Definición del Problema

Las aplicaciones bancarias gestionan operaciones críticas que exigen una implementación robusta, segura y transaccionalmente consistente. El desarrollo de una API bancaria de referencia supone afrontar problemas técnicos reales presentes en la industria financiera:

- Consistencia de saldos: dos operaciones concurrentes sobre la misma cuenta pueden corromper el balance si no se aplica aislamiento de transacciones.
- Seguridad de credenciales: almacenar contraseñas en texto plano o exponer datos de usuario por defecto constituye un riesgo grave.
- Integridad referencial: la eliminación de cuentas con saldo activo o la modificación directa del balance sin pasar por el flujo transaccional pueden generar inconsistencias en la base de datos.
- Trazabilidad: cualquier movimiento financiero debe quedar registrado con origen, destino, importe y tipo para garantizar la auditoría.

MazeBank API nace como una solución backend de referencia que aborda todos estos retos de forma completa, reproducible y documentada.

1.2 Estado del Arte y Justificación

Antes de definir la arquitectura se analizaron alternativas y enfoques existentes:

Alternativa	Limitaciones	Por qué MazeBank lo mejora
BaaS genéricos (Plaid, Stripe)	Dependen de terceros, costes de integración y opacidad interna.	Implementación propia: control total del flujo y la lógica de dominio.
Demos educativas en Express	Sin transacciones SQL, sin control de concurrencia, sin documentación.	Transacciones atómicas, bloqueo de filas y Swagger completo.
APIs REST genéricas (CRUD simple)	No modelan restricciones de dominio bancario ni integridad de saldo.	Hooks de Sequelize y validaciones de negocio en capa de servicio.

MazeBank API cubre el hueco de una API bancaria de referencia: organizada por responsabilidades (MVC), con despliegue reproducible mediante Docker Compose, documentación interactiva Swagger y pruebas manuales de los flujos críticos.

2 Análisis de Requisitos

Para delimitar el alcance del MVP se establecieron los siguientes requisitos formales que guiaron el desarrollo.

2.1 Requisitos Funcionales (RF)

ID	Nombre	Descripción
RF-01	Registro de usuario	El sistema permite registrar un usuario con DNI único, contraseña cifrada, nombre, apellidos, email y teléfono.
RF-02	Autenticación JWT	El usuario puede iniciar sesión con DNI y contraseña y recibe un token JWT con expiración de 1 hora.
RF-03	Gestión de cuentas	El usuario autenticado puede crear, consultar, renombrar y eliminar sus cuentas bancarias. El número de cuenta (ES000001...) se genera automáticamente.
RF-04	Depósito	El usuario puede ingresar fondos en una cuenta propia. La operación se registra como transacción de tipo DEPOSIT.
RF-05	Retirada	El usuario puede retirar fondos siempre que el saldo sea suficiente. Registrada como WITHDRAW.
RF-06	Transferencia	El usuario puede transferir fondos entre dos cuentas cualquiera. Registrada como TRANSFER con cuenta origen y destino.
RF-07	Historial paginado	Consulta del historial de movimientos de una cuenta con paginación y filtro por tipo de transacción.
RF-08	Gestión de perfil	El usuario puede consultar, actualizar y eliminar su propio perfil.

2.2 Requisitos No Funcionales (RNF)

ID	Categoría	Descripción
RNF-01	Seguridad	Las contraseñas se almacenan cifradas con bcrypt. El modelo Users excluye el campo password por defecto en todas las consultas.
RNF-02	Autenticación	Todas las rutas de cuentas y transacciones requieren Authorization: Bearer <token> válido con expiración de 1 hora.
RNF-03	Integridad	El balance solo puede modificarse mediante operaciones transaccionales (BEGIN / COMMIT / ROLLBACK). Los hooks de Sequelize bloquean modificaciones directas sobre balance y accountNumber.
RNF-04	Consistencia	Las cuentas implicadas en una operación se consultan con bloqueo de fila (SELECT ... FOR UPDATE) para evitar condiciones de carrera.
RNF-05	Restricciones	Una cuenta con saldo distinto de cero no puede eliminarse. Se valida en capa de servicio antes de ejecutar el DELETE.
RNF-06	Rate limiting	Límite de 100 peticiones cada 5 minutos por IP para mitigar abuso básico.

RNF-07	Reproducibilidad	El proyecto arranca con un único comando (docker compose up --build). La base de datos se inicializa automáticamente con schema y datos de ejemplo.
RNF-08	Documentación	Todos los endpoints están documentados con Swagger/OpenAPI accesible en /api-docs. La documentación interna se genera con JSDoc.

3 Planificación y Gestión del Tiempo

Se estimó una capacidad de 6 horas efectivas diarias de desarrollo, reservando tiempo para documentación y resolución de bloqueos.

3.1 Desglose de Tareas (Backlog)

Fase	Tarea	Descripción técnica	Estimación
Diseño	Modelado E-R y contrato API	Diagrama de entidades (Users, Accounts, Transactions), definición de endpoints y esquema SQL.	6 h
Backend	Setup inicial	Configuración de Express 5, CORS, dotenv, conexión Sequelize a PostgreSQL y Docker Compose.	4 h
Backend	Sistema de autenticación	Endpoints /register y /login, middleware JWT, hash bcrypt y defaultScope en Users.	8 h
Backend	CRUD de cuentas	Servicios y controladores para crear, listar, actualizar y eliminar cuentas. Generación automática de número ES-format. Hooks de Sequelize.	10 h
Backend	Lógica transaccional	Depósito, retirada y transferencia con transacciones SQL atómicas, bloqueo de filas y validaciones de negocio.	14 h
Backend	Historial paginado	Endpoint GET /api/transaction/:accountNumber con paginación y filtro por tipo.	4 h
Frontend	Vista EJS + CSS	Pantalla de inicio home.ejs con layout principal, componentes header/footer y estilos CSS.	4 h
Docs	Swagger + JSDoc	Decoración de todos los endpoints con anotaciones OpenAPI 3.0. Configuración jsdoc.json y generación de docs/.	6 h
Cierre	QA y memoria	Pruebas manuales de flujos críticos, corrección de bugs y redacción de documentación técnica.	8 h

Se aplicó un factor de seguridad del 20% sobre las horas estimadas para cubrir imprevistos. El total estimado asciende a aproximadamente 64 horas efectivas de desarrollo.

3.2 Cronograma de Trabajo (Gantt)

Las fases de desarrollo se organizaron de forma secuencial, respetando las dependencias entre módulos:

Tarea	Día 1-2	Día 3-4	Día 5-6	Día 7-8	Día 9	Día 10
Diseño y modelado	■					
Setup + Auth		■				
CRUD cuentas			■			
Lógica transaccional			■	■		
Vista EJS + Swagger				■		
QA + Memoria					■	■

4 Análisis Funcional y Flujos

MazeBank API expone un único tipo de actor (el usuario autenticado) que puede gestionar sus propias cuentas y ejecutar operaciones financieras. A continuación se describen los flujos principales del sistema.

4.1 Flujo de Transferencia (Diagrama de Secuencia)

El flujo de transferencia es el más crítico del sistema al involucrar dos cuentas, validaciones de saldo, propiedad y consistencia transaccional:

```
Cliente → POST /api/transaction/transfer
API → Middleware JWT: Validar Bearer token
Middleware → API: Usuario autenticado
API → TransactionService: createTransfer()
Service → DB: BEGIN TRANSACTION
Service → DB: SELECT ... FOR UPDATE (cuenta origen y destino)
Service: Validar saldo suficiente y propiedad de la cuenta
Service → DB: UPDATE balance origen y destino
Service → DB: INSERT INTO Transactions
Service → DB: COMMIT
API → Cliente: 201 Created { transacción }
```

4.2 Modelo de Datos (Entidad-Relación)

El modelo relacional se compone de tres entidades principales con las siguientes relaciones:

- Un Users puede tener múltiples Accounts (1:N).
- Una Accounts puede aparecer como cuenta origen (senderAccountId) en múltiples Transactions (1:N).
- Una Accounts puede aparecer como cuenta destino (receiverAccountId) en múltiples Transactions (1:N).

Users	Accounts	Transactions
id (PK)	id (PK)	id (PK)
dni (UNIQUE)	userId (FK → Users)	senderAccountId (FK → Accounts)
password (hashed)	accountName	receiverAccountId (FK → Accounts)
firstName	accountNumber (UNIQUE, ES000001...)	amount (DECIMAL)
lastName	balance (DECIMAL)	details
email		type (DEPOSIT WITHDRAW TRANSFER)
phoneNumber		createdAt / updatedAt

5 Arquitectura y Modelo de Datos

5.1 Arquitectura MVC

El proyecto sigue el patrón MVC con separación estricta entre capas. Cada capa tiene una responsabilidad única:

- **Routes (Express):** Definen los endpoints y aplican el middleware de autenticación. No contienen lógica de negocio.
- **Controllers:** Reciben la petición HTTP, extraen parámetros, invocan el servicio correspondiente y devuelven la respuesta.
- **Services:** Contienen toda la lógica de negocio: validaciones, construcción de queries, transacciones SQL y reglas de dominio.
- **Models (Sequelize):** Mapean las tablas de PostgreSQL y definen hooks para proteger campos críticos.

El flujo de una petición sigue siempre el mismo camino: Cliente → Router → Middleware JWT → Controller → Service → Model → PostgreSQL.

5.2 Diseño de la API (Endpoints Principales)

Método	Ruta	Descripción	Auth JWT
POST	/api/user/register	Registrar nuevo usuario.	No
POST	/api/user/login	Autenticar y obtener token JWT.	No
GET	/api/user/me	Consultar perfil del usuario autenticado.	Sí
PATCH	/api/user/me	Actualizar datos del perfil.	Sí
DELETE	/api/user/me	Eliminar cuenta de usuario.	Sí
GET	/api/account	Listar todas las cuentas del usuario.	Sí
POST	/api/account	Crear nueva cuenta bancaria.	Sí
GET	/api/account/:accountNumber	Consultar detalle de una cuenta.	Sí
PATCH	/api/account/:accountNumber	Renombrar cuenta bancaria.	Sí
DELETE	/api/account/:accountNumber	Eliminar cuenta (solo si saldo es cero).	Sí
GET	/api/transaction/:accountNumber	Historial paginado de movimientos.	Sí
POST	/api/transaction/deposit	Realizar un depósito.	Sí
POST	/api/transaction/withdraw	Realizar una retirada.	Sí
POST	/api/transaction/transfer	Realizar una transferencia entre cuentas.	Sí

5.3 Stack Tecnológico y Justificación

Tecnología	Capa	Justificación
Node.js 20+	Runtime	Arquitectura no bloqueante (event loop) adecuada para APIs con múltiples peticiones concurrentes.
Express 5	Framework HTTP	Versión más reciente con soporte nativo para async/await en middlewares y manejo de errores simplificado.
PostgreSQL	Base de datos	SQL elegido sobre NoSQL por la necesidad de integridad referencial (FK), transacciones ACID y control de concurrencia con bloqueo de filas.
Sequelize ORM	ORM	Abstracción del acceso a datos con soporte para transacciones, hooks de modelo y migraciones. Reduce SQL repetitivo sin perder control cuando se necesita.
JWT	Autenticación	Stateless: no requiere sesión en servidor. Expiración corta (1 hora) limita el impacto de tokens comprometidos.
bcrypt	Seguridad	Hash adaptativo con salt aleatorio. Las contraseñas nunca se almacenan en texto plano.
Docker Compose	DevOps	Levanta API + PostgreSQL + pgAdmin en un único comando. Garantiza reproducibilidad en cualquier entorno.
Swagger/OpenAPI	Documentación	Documentación interactiva que permite probar la API directamente desde el navegador sin herramientas externas.
EJS	Vistas	Motor de plantillas ligero integrado en Express para servir la vista inicial. No requiere bundler ni SPA.

6 Retos Técnicos y Soluciones

6.1 Reto: Transacciones Financieras Atómicas

Situación: Las operaciones de depósito, retirada y transferencia modifican el balance de una o varias cuentas. Sin transacciones SQL, un fallo a mitad de la operación (p. ej. red caída tras actualizar la cuenta origen pero antes de actualizar la destino) dejaría la base de datos en estado inconsistente.

Tarea: Garantizar que cualquier operación financiera se completa íntegramente o no se aplica en absoluto (atomicidad).

Acción: Se implementó cada operación financiera dentro de una transacción Sequelize con nivel de aislamiento SERIALIZABLE y bloqueo explícito de filas:

```
const t = await sequelize.transaction();
try {
  // Bloqueo de filas para evitar condiciones de carrera
  const sender = await Account.findOne({
    where: { accountNumber: senderAccountNumber },
    lock: t.LOCK.UPDATE, transaction: t
  });
  // Validar saldo y propiedad, actualizar balances, registrar transacción
  await t.commit();
} catch (err) {
  await t.rollback(); // Revertir todos los cambios
  throw err;
}
```

Resultado: Las operaciones financieras son atómicas. Cualquier fallo provoca un ROLLBACK automático, preservando la integridad de los saldos. El bloqueo de filas (LOCK.UPDATE) elimina las condiciones de carrera en operaciones concurrentes sobre la misma cuenta.

6.2 Reto: Protección del Balance con Hooks de Sequelize

Situación: Con Sequelize es posible actualizar directamente el campo balance de una cuenta mediante `account.update({ balance: newValue })` desde cualquier parte del código, saltándose las validaciones del servicio.

Tarea: Garantizar que el balance únicamente cambie a través del flujo transaccional controlado, previniendo modificaciones accidentales o intencionadas.

Acción: Se implementaron hooks `beforeUpdate` y `beforeCreate` en el modelo `Account` que detectan si el campo balance ha sido modificado fuera del contexto transaccional y lanzan un error:

```
Account.addHook('beforeUpdate', (account) => {
  if (account.changed('balance')) {
```

```
        throw new Error('El balance solo puede modificarse mediante  
transacciones.');
```

Resultado: Cualquier intento de modificar el balance directamente lanza una excepción, forzando el uso del servicio transaccional. Esto también protege el campo accountNumber (formato ES000001) de ser modificado una vez asignado.

7 Pruebas de Calidad (QA)

Se realizó una batería de pruebas manuales sobre los flujos críticos del sistema para validar tanto el camino feliz (Happy Path) como los casos límite (Edge Cases).

Caso de Prueba	Resultado Esperado	Herramienta	Estado
Registro con DNI duplicado	El sistema debe devolver 409 Conflict. El DNI debe ser único.	Swagger UI	✓
Login con contraseña incorrecta	Devuelve 401 Unauthorized sin revelar si el DNI existe en el sistema.	Swagger UI	✓
Acceso a ruta protegida sin token	La API devuelve 401 Unauthorized. El middleware rechaza la petición antes de llegar al controlador.	curl	✓
Token expirado (>1 hora)	La API devuelve 401. El middleware detecta la expiración del JWT y rechaza.	Postman	✓
Retirada con saldo insuficiente	Devuelve 400 Bad Request con mensaje de saldo insuficiente. El balance no cambia.	Swagger UI	✓
Transferencia entre cuentas válidas	Devuelve 201 Created. El balance origen decrece y el destino aumenta en el importe exacto.	Swagger UI + pgAdmin	✓
Eliminar cuenta con saldo > 0	Devuelve 400 Bad Request. La cuenta no se elimina hasta que el saldo sea cero.	Swagger UI	✓
Modificar balance directamente (hook)	El hook beforeUpdate de Sequelize lanza un error y la operación es rechazada.	Script Node.js	✓
Rate limiting (>100 req / 5 min)	La API devuelve 429 Too Many Requests. Las siguientes peticiones se bloquean hasta que transcurra el intervalo.	Script ab (Apache Bench)	✓

8 Conclusiones y Futuros Pasos

El desarrollo de MazeBank API ha permitido profundizar en los aspectos más críticos de una API bancaria real: atomicidad transaccional, seguridad de credenciales, integridad de datos y documentación reproducible.

Los objetivos del MVP se han cumplido íntegramente: registro y autenticación de usuarios, gestión completa de cuentas, operaciones financieras atómicas con registro de movimientos, documentación interactiva Swagger y despliegue reproducible con Docker Compose.

Entre las principales lecciones técnicas extraídas:

- La importancia de las transacciones SQL ACID en cualquier sistema que gestione dinero o recursos críticos.
- El uso de hooks de ORM como segunda línea de defensa para proteger invariantes de dominio.
- La necesidad de documentar los endpoints desde el inicio del desarrollo, no como tarea de cierre.

Mejoras para la v2.0

1. Implementar refresh tokens para mejorar la experiencia de usuario sin comprometer la seguridad.
2. Añadir pruebas de integración automatizadas con Jest y Supertest sobre los flujos transaccionales.
3. Notificaciones en tiempo real (WebSockets) para avisar al usuario cuando recibe una transferencia.
4. Exportación del historial de movimientos a CSV/PDF.
5. Dashboard frontend React que consuma la API y visualice saldos y movimientos de forma gráfica.
6. CI/CD con GitHub Actions para ejecutar tests y validar la build en cada pull request.